SESIÓN TEÓRICA Y LABORATORIO PRÁCTICO

Análisis de Tramas en Capa 2, Desencapsulamiento TCP/IP y Raw Sockets

Del Cliente-Servidor a la intercepción pasiva de paquetes en C (sniffer.c)

Asignatura: Arquitectura Cliente-Servidor

1. De los Sockets de Aplicación a los Raw Sockets

El Mito en Arquitectura Cliente-Servidor

- **Abstracción del programador:** Al usar SOCK_STREAM sobre AF_INET, parece existir un 'tubo directo' entre Cliente y Servidor.
- **Transparencia del Kernel:** El sistema operativo agrega y recorta las cabeceras TCP, IP y Ethernet automáticamente.
- **Filtrado por Hardware (NIC):** La tarjeta de red descarta tramas cuya MAC destino no sea la suya (salvo Broadcast/Multicast).

La Realidad en Redes y Ciberseguridad

- **Modo Promiscuo:** Ordena al driver de la NIC entregar TODAS las tramas del cable/aire al CPU sin filtrar por MAC.
- **Raw Sockets (AF_PACKET + SOCK_RAW):** Evitan que el Kernel desarme el paquete; recibimos la trama binaria íntegra desde Capa 2.
- **Privilegios de Superusuario:** Abrir sockets crudos requiere permisos root (CAP_NET_RAW) por seguridad.

PASO 1 EN SNIFFER.C: CREACIÓN DEL SOCKET EN CRUDO (CAPA 2)

```
raw_socket = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
if (raw_socket < 0) { perror("Error (¿Ejecutaste con sudo?)"); return 1; }
```

2. Encapsulamiento y Aritmética de Punteros en C

El Buffer de 64 KB y Desplazamientos

- **Reserva de Memoria:** malloc(65536) reserva 64 KB (tamaño máximo teórico de un paquete IP de 16 bits: 2^16 = 65,536 B).
- **Captura con recvfrom():** Copia los bits crudos de la NIC al arreglo unsigned char *buffer.
- **Type Casting y Offsets:** Superponemos estructuras (ethhdr, iphdr, tcphdr) sumando desplazamientos al apuntador base buffer.
- **Secuencia Contigua:** [Ethernet 14B] -> [IPv4 20-60B] -> [TCP 20-60B] -> [Payload Aplicación].

2. Orden de Bytes en la Red (Endianness)

Network Byte Order (Big-Endian)

- **Estándar TCP/IP:** Los protocolos de red transmiten primero el byte más significativo (MSB).
- **Ejemplo Puerto 4898 (0x1322):** En el cable viaja primero el byte 0x13 y después el byte 0x22.
- **Función htons():** Host to Network Short — se usa al enviar parámetros de 16 bits al socket (ej. htons(ETH_P_ALL)).

Host Byte Order (Little-Endian)

- **Arquitectura x86 / x86_64:** Los procesadores modernos guardan primero el byte menos significativo (LSB) en RAM.
- **Riesgo de inversión:** Si leemos 0x1322 directo de la red sin convertir, la CPU interpreta 0x2213 (puerto 8723 en decimal).
- **Función ntohs():** Network to Host Short — indispensable al leer h_proto, tcph->source y tcph->dest.

IMPACTO PRÁCTICO EN SNIFFER.C

```
// Conversión de Big-Endian (Red) a Little-Endian (CPU) con ntohs():
if (ntohs(eth->h_proto) == ETH_P_IP) { ... }                    // 0x0800 -> IPv4
if (ntohs(tcph->dest) == 4898 || ntohs(tcph->source) == 4898)   // Puerto del laboratorio
```

3A. Capa 2 (Enlace de Datos): Cabecera Ethernet II

Estructura struct ethhdr (14 Bytes Fijos)

- **Cabecera <netinet/if_ether.h>:** Se ubica exactamente en el byte 0 de nuestro buffer.
- **MAC Destino (h_dest):** 6 bytes (48 bits) de la interfaz receptora.
- **MAC Origen (h_source):** 6 bytes (48 bits) impresos en hexadecimal (%02x).
- **EtherType (h_proto):** 2 bytes que identifican el protocolo de Capa 3. ETH_P_IP (0x0800) indica IPv4.

PASO 3 EN SNIFFER.C: LECTURA DE TRAMA ETHERNET

```
struct ethhdr *eth = (struct ethhdr *)buffer; // Apunta al byte 0 (primeros 14 bytes)
if (ntohs(eth->h_proto) == ETH_P_IP) {
    printf("MAC Origen: %02x:%02x:%02x:%02x:%02x:%02x", eth->h_source[0], ...);
}
```

3B y 3C. Capas de Red (IPv4) y Transporte (TCP)

Longitudes Dinámicas (IHL y Data Offset)

- **Salto a IPv4 (+14 bytes):** buffer + sizeof(struct ethhdr) nos posiciona al inicio de struct iphdr.
- **Campo IHL (iph->ihl):** Mide la cabecera IP en palabras de 32 bits (4 bytes). Multiplicamos iph->ihl * 4 (ej. 5 * 4 = 20 B).
- **Protocolo (iph->protocol == 6):** El estándar IANA asigna el 6 a TCP (1 = ICMP, 17 = UDP).
- **Salto a TCP (+14 + iphdrlen):** Posiciona struct tcphdr justo después de IPv4.
- **Campo Data Offset (tcph->doff):** Tamaño de cabecera TCP en bloques de 4 bytes: tcp_header_len = tcph->doff * 4.
- **Filtro de Puertos:** Capturamos peticiones y respuestas en el puerto 4898.

4. Extracción del Payload y Riesgo del Texto Plano

Cálculo Exacto del Payload (Capa 7)

- **Suma de Cabeceras:** total_headers_size = 14 (Ethernet) + iphdrlen (IP) + tcp_header_len (TCP).
- **Puntero a Datos:** payload = buffer + total_headers_size apunta al primer carácter del mensaje.
- **Filtrado de Paquetes de Control:** Si payload_size == 0, es un paquete SYN, ACK o FIN sin datos de usuario; lo ignoramos.

Impacto en Ciberseguridad (Confidencialidad)

- **Vulnerabilidad del Texto Plano:** Diseñar arquitecturas Cliente-Servidor sobre TCP puro expone contraseñas y comandos en la red local.
- **Prevención de Buffer Over-read:** Usamos el especificador %.*s pasando payload_size para imprimir solo los bytes exactos del mensaje.
- **Mitigación:** Implementar cifrado en tránsito mediante TLS (Transport Layer Security).

PASO 6 EN SNIFFER.C: AISLAMIENTO E IMPRESIÓN SEGURA DEL PAYLOAD

```
int tcp_header_len = tcph->doff * 4;
int total_headers_size = sizeof(struct ethhdr) + iphdrlen + tcp_header_len;
unsigned char *payload = buffer + total_headers_size;
int payload_size = data_size - total_headers_size;
if (payload_size > 0) { printf("Contenido (Payload):\n%.*s\n", payload_size, payload); }
```

5. Laboratorio Práctico: Ejecución en 3 Terminales

TERMINAL 1: COMPILAR Y ACTIVAR SNIFFER (ROOT)

```
$ gcc sniffer.c -o sniffer
$ sudo ip link set eth0 promisc
on
$ sudo ./sniffer

>>> SNIFFER ACTIVO...
>>> Esperando tráfico TCP
    en el puerto 4898...
```

TERMINAL 2: SERVIDOR TCP EN PUERTO 4898

```
$ nc -lvp 4898
Listening on 0.0.0.0 4898
Connection received on
127.0.0.1 54322
Hola servidor, enviando
contrasena_secreta_123
```

TERMINAL 3: CLIENTE ENVIANDO TEXTO PLANO

```
$ nc 127.0.0.1 4898
Hola servidor, enviando
contrasena_secreta_123
[Enter]
```

Resultado Observado en la Terminal 1 (Intercepción Exitosa)

- **Cabeceras Desencapsuladas:** MAC Origen: 00:00:00:00:00:00 | IP Origen: 127.0.0.1 | Puerto Src: 54322 -> Puerto Dst: 4898
- **Payload Capturado:** Se visualiza íntegramente la cadena 'Hola servidor, enviando contrasena_secreta_123' sin alterar la sesión Cliente-Servidor.

Referencias Bibliográficas y Estándares IETF

Bibliografía Académica y RFCs Oficiales

- **Comer, D. E. (2013):** Internetworking with TCP/IP Volume 1: Principles, Protocols, and Architecture (6th ed.). Addison-Wesley.
- **Kerrisk, M. (2010):** The Linux Programming Interface: A Linux and UNIX System Programming Handbook. No Starch Press.
- **Linux Programmer's Manual:** packet(7) - packet interface on device level (AF_PACKET, SOCK_RAW). https://man7.org/linux/man-pages/man7/packet.7.html
- **Postel, J. (1981) — RFC 791:** Internet Protocol: DARPA Internet Program Protocol Specification (STD 5). IETF. https://datatracker.ietf.org/doc/html/rfc791
- **Eddy, W. (Ed.) (2022) — RFC 9293:** Transmission Control Protocol (TCP) (STD 7, actualiza RFC 793). IETF. https://datatracker.ietf.org/doc/html/rfc9293
- **Stevens, W. R., Fenner, B., & Rudoff, A. M. (2003):** UNIX Network Programming, Vol. 1: The Sockets Networking API (3rd ed., Cap. 28: Raw Sockets). Addison-Wesley.
- **Tanenbaum, A. S., Feamster, N., & Wetherall, D. J. (2021):** Computer Networks (6th ed.). Pearson.